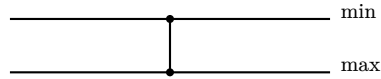


The network djbsort's AVX2 sort runs, at 64 lines

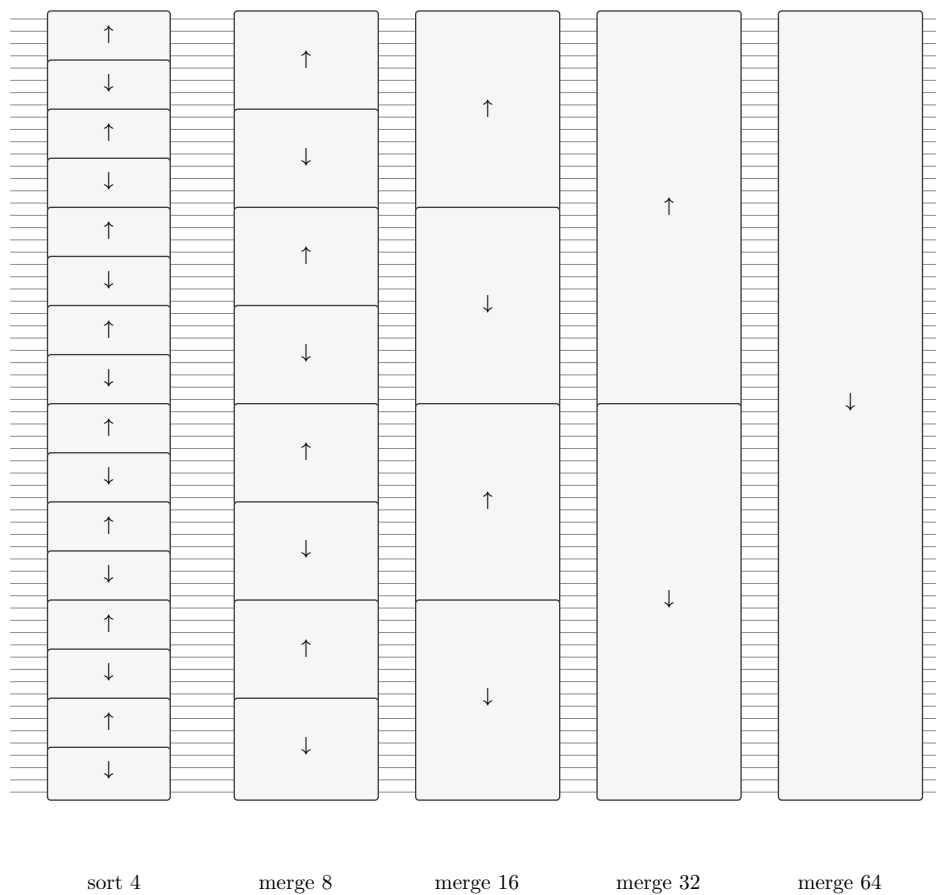
Laurent Théry

A connector joins two lines; the minimum leaves on the upper one.



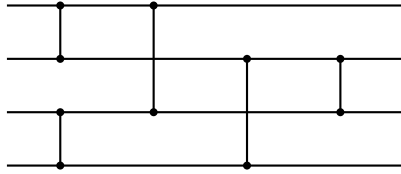
At 64 lines the sort performs 656 comparisons, in 82 groups of eight done at once by one vector instruction. Drawn one comparison at a time the picture is unreadable, so each box below stands for a whole component, and its arrow points towards the larger values.

The shape is a bitonic sort. First every group of four lines is sorted, the upper four decreasing and the lower four increasing, so every group of eight falls and then rises. Then merges of doubling size, each turning two neighbouring runs of opposite direction into one.

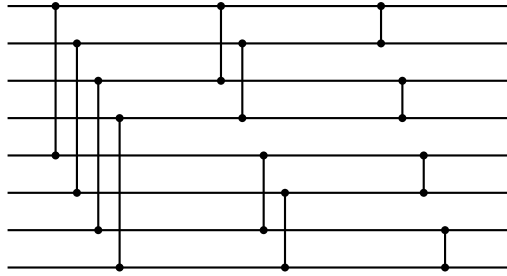


What the boxes contain

sort 4 is the five-comparison network on four lines, at distances 1, 1, 2, 2, 1. A bitonic sort of four would use six, so this is where the code is smaller than a plain bitonic sorter.

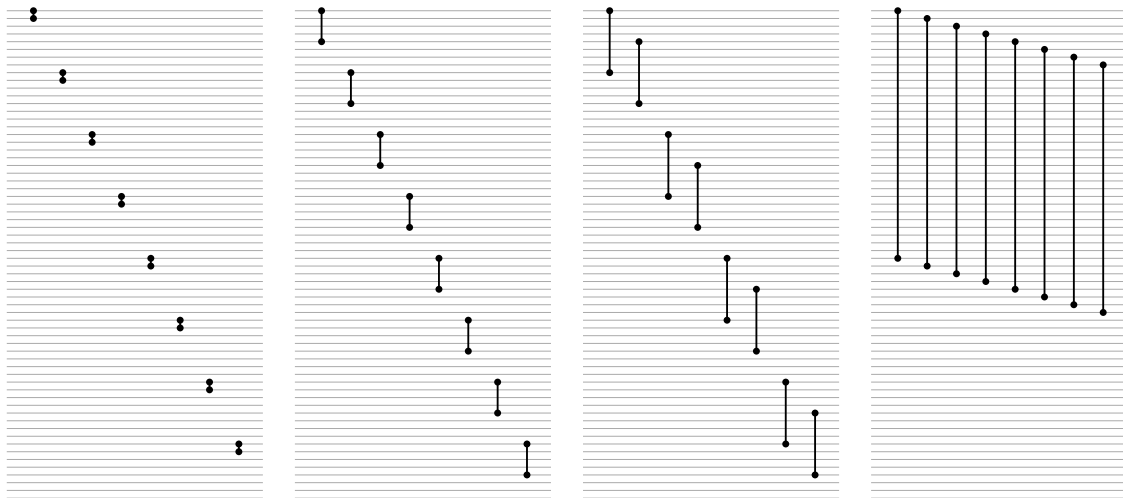


merge $2m$ takes a run down followed by a run up and returns one sorted run. It is the usual cascade: compare at distance m , then $m/2$, down to 1. At 64 lines the merges use distances 4, 2, 1 then 8, 4, 2, 1 then 16, 8, 4, 2, 1 then 32, 16, 8, 4, 2, 1 — which is what the code performs, once its comparisons are named by the array position each value ends in.



Where the eight-at-a-time grouping sits

One vector instruction performs eight comparisons at once. They are always eight copies of the same comparison, placed regularly across the array, so a single instruction is a connector in its own right — its eight comparisons are on sixteen distinct lines. Here are four of the eighty-two, on all 64 lines.



no. 1, distance 1

no. 11, distance 4

no. 23, distance 8

no. 59, distance 32

The comparisons of one instruction are eight lines apart in most stages, and next to each other in the stage that follows the transpose. That is what the transposes are for: they change which sixteen lines sit together in the registers, so that the next distance can again be handled eight at a time.

The whole sort is 82 such instructions, and the picture on the first page is what they add up to.

What the transposes do

A transpose compares nothing, so it does not appear in the picture on the first page at all: naming each line by the position its value ends in absorbs it. What it decides is which eight positions sit in one register, and hence which comparisons can be done together.

Write the 64 positions as an eight by eight grid, one register to a row. On the left each register holds eight consecutive positions; after the transpose each holds eight positions that are eight apart. The shaded register becomes a column.

0	1	2	3	4	5	6	7		0	8	16	24	32	40	48	56
8	9	10	11	12	13	14	15		1	9	17	25	33	41	49	57
16	17	18	19	20	21	22	23		2	10	18	26	34	42	50	58
24	25	26	27	28	29	30	31		3	11	19	27	35	43	51	59
32	33	34	35	36	37	38	39		4	12	20	28	36	44	52	60
40	41	42	43	44	45	46	47		5	13	21	29	37	45	53	61
48	49	50	51	52	53	54	55		6	14	22	30	38	46	54	62
56	57	58	59	60	61	62	63		7	15	23	31	39	47	55	63

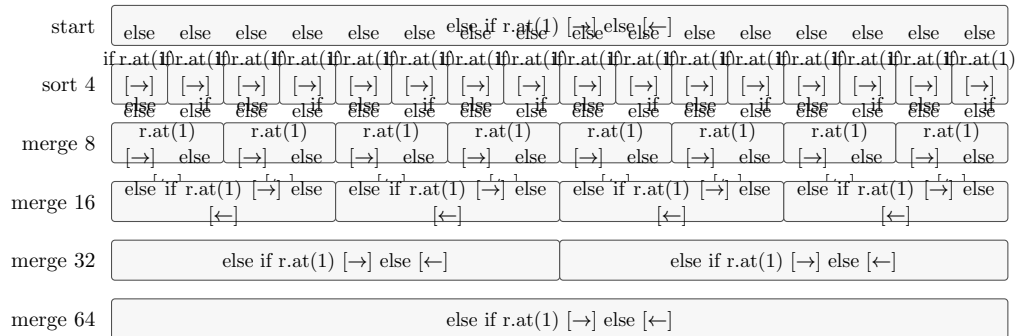
transpose
→

one register to a row — before, and after

That is why the same distance is handled with two different pictures on the previous page: comparisons eight lines apart while the registers hold consecutive positions, comparisons on neighbouring lines once they hold spread-out ones. The code transposes twice, and between the two it does the merges that need the other grouping.

How it works, at 64 positions

The same thing seen on the data rather than on the network. A bar is the 64 array positions; a box inside it is a sorted run, the arrow pointing towards the larger values. Each stage joins neighbouring runs of opposite direction into runs of twice the length.



the first bar is the input, unsorted; the last is the answer

Two runs of opposite direction laid end to end form a sequence that rises then falls, or falls then rises, and that is exactly what a merge needs. So the alternation is not decoration: it is what lets the next stage be a merge rather than a full sort.

The eight-at-a-time part sits underneath all of this. Every box in a bar is handled together with seven boxes like it elsewhere in the same bar, which is why every run at a given stage has the same length: the eight are the eight lanes of one instruction.